

llm-bridging

Phase 0 Research — Grounded Question Answering with Citations, Bridged to Local or External Models

Feature: 001-workshop-curriculum-platform **Covers:** User Story 4; FR-021 ... FR-025; SC-009, SC-010. Touches FR-032/FR-033/FR-040 (evidence) and FR-037 (passage identity). **Measured on:** 2026-08-31, on the operator's machine, while a lumen index --force rebuild and a 287-gate sweep were running. **Author's standing constraint:** every claim about what is installed or reachable below is followed by the command that produced it and the real output. Anything not measured is marked **UNVERIFIED** with the reason.

ROUTE NAMES UPDATED 2026-09-01, throughout this document. Where this research said `/api/answer` and `/api/answering/status` it now says `/api/ask` and `/api/ask/status`, in six places. The names are the *only* thing that changed: the separate-route-trees argument, the three-state table, the locality invariant and gate design in §3 and §5 are untouched, and none of them ever depended on the spelling. The implementation mounts `/api/ask`; the contract was amended to match on 2026-09-01, and this document follows it rather than preserving a name that never existed in code. The decision, its evidence and its counter-argument are recorded once, in [contracts/http-api.md](#) §3.10, and are not restated here.

Executive summary — the three facts that shape this design

1. **There is no generative model on this machine.** ollama is up and healthy, but it serves exactly two models and both are *embedding* models. Question answering (US4) cannot run today at all, with any design. It is blocked on a deliberate operator action (downloading a generative model), not on code.
 2. **“HelixLLM / HelixAgent bridging” does not exist in this repository.** It exists as *marketing copy* in `_content*/products/` and `_analysis/content-briefs/`, and as environment variables in `~/.bashrc` that point at a checkout **that is not present on this disk**. One real, reusable artefact was found — the LLMProvider Go module, checked out under a *different* project — and it is not a dependency of `vasic`.
 3. **CPU-only generation on this box is measured in tens of seconds per answer, not milliseconds.** The operator’s phrase “getting instant answers” and this hardware are in direct conflict. The honest number is below in §6. US1–US3 must not be sequenced behind it.
-

1. What local inference is actually available?

Decision

Treat the ollama instance as an embedding-only service. Assume ZERO generative capability until an operator installs a model. Design US4 so that its default, out-of-the-box state is answering unavailable — a first-class, honestly-reported state, not an error.

Evidence

```
curl -s --max-time 10 http://localhost:11434/api/tags
curl -s --max-time 10 http://localhost:11434/api/ps
curl -s --max-time 5 http://localhost:11434/api/version
```

Real output (`/api/tags`, reformatted for reading — values verbatim):

name	family	parameter_size	quantization_level	size	modified_at
jina-					
embeddings-		160.28M	F16		2026-08-27T00:36:5

name	family	parameter_size	quantization_level	size	modified_at
code-cpu:latest	jina-			323	
	bert-			008	
	v2			168	
ordis/jina-embeddings-v2-base-code:latest	jina-bert-v2	160.28M	F16	323008156	2026-08-26T20:37:5

/api/ps (currently resident):

```
{
  "models": [
    {
      "name": "ordis/jina-embeddings-v2-base-code:latest",
      "parameter_size": "160.28M",
      "quantization_level": "F16",
      "context_length": 8192,
      "size_vram": 0,
      "expires_at": "2026-08-31T22:21:41+02:00"
    }
  ]
}
```

/api/version → {"version": "0.23.4"}

Both entries are the same model. jina-bert-v2 is a BERT-family encoder; jina-embeddings-v2-base-code is an embedding model with no LM head. jina-embeddings-code-cpu is a locally re-tagged copy of it (identical parameter_size, near-identical byte size, one day apart). Neither can generate text. There is **no** llama, qwen, phi, gemma, mistral, or any decoder-only family present.

Corroborating measurement — the whole model store is 309 MB across 7 blobs, which cannot hold a generative model:

```
du -sh /var/lib/ollama/.ollama/models # → 309M
ls /var/lib/ollama/.ollama/models/blobs | wc -l # → 7
```

size_vram: 0 confirms the CPU-only path is in force at runtime, not merely configured.

No other inference backend is running. LM Studio (which the repository's own tooling already supports as an alternative backend — see §2) is not up:

```
curl -s --max-time 3 http://localhost:1234/v1/models # → empty,
no listener
ss -ltn | grep -E ':(11434|1234|8080)'
# LISTEN 127.0.0.1:11434
# LISTEN 127.0.0.1:8080 (node, unrelated)
```

CPU-only enforcement is real and verified

```
systemctl show ollama -p EnvironmentFiles --value
# → /etc/sysconfig/ollama (ignore_errors=yes)
grep -v '^s*#' /etc/sysconfig/ollama
# → GGML_VK_VISIBLE_DEVICES=-1
journalctl -u ollama --no-pager | grep -oE 'library=[a-zA-Z]+' |
    tail -3
# → library=cpu
# → library=cpu
# → library=cpu
```

This is not incidental. scripts/ollama-vulkan-remediation.sh:7 records the reason: “ollama running library=Vulkan on an Intel iGPU silently corrupts embeddings.” The CPU-only setting is a correctness remedy for a real, previously-observed data-corruption fault. **Do not plan around re-enabling the GPU.**

The hardware

```
lscpu | grep -E '^(Model name|CPU(s)|Core|Thread)'
# Model name: 11th Gen Intel(R) Core(TM) i7-1165G7 @ 2.80GHz
# CPU(s): 8   Core(s) per socket: 4   Thread(s) per core: 2
lscpu | grep -oE 'avx[0-9_a-z]*' | sort -u
# avx avx2 avx512f avx512bw avx512dq avx512vl avx512_vnni
    avx512_vbmi ...
free -g   # total 62 Gi, available 53 Gi
df -h /   # 170G available
```

Four physical Tiger Lake cores with AVX-512 (including VNNI, which llama.cpp uses for int8 GEMM), LPDDR4x memory, 53 GiB RAM free, 170 GiB disk free. **RAM and disk are not the constraint. Memory bandwidth and four cores are.**

What would be needed (NOT executed — hard constraint)

No ollama pull was run. For the record, the operator action required to unblock US4:

Candidate	Q4_K_M size	Why considered
qwen2.5:1.5b-instruct-q4_K_M	~1.0 GB	Fastest usable; weakest instruction-following (see §4 risk)
	~2.0 GB	

Candidate	Q4_K_M size	Why considered
llama3.2:3b-instruct-q4_K_M		Best speed/obedience balance for this box
qwen2.5:3b-instruct-q4_K_M	~1.9 GB	Comparable; stronger structured-output adherence
qwen2.5:7b-instruct-q4_K_M	~4.7 GB	Best refusal behaviour; ~2.5× slower (§6)

Recommendation to the operator: `llama3.2:3b-instruct-q4_K_M` or `qwen2.5:3b-instruct-q4_K_M` as the first model to try, `qwen2.5:7b-instruct-q4_K_M` if SC-010 fails at 3B. The download must be done when the indexing backend is idle — see the queue-contention measurement in §6.

UNVERIFIED: the four sizes above are the widely-published quantised sizes for these models; none was downloaded or measured here, because pulling one is forbidden by this research task’s constraints and would saturate a backend that is currently busy.

Rationale

Assuming a chat model exists because ollama exists is precisely the “confident but wrong” failure the governing constitution forbids. The measurement says the opposite of the assumption. Building US4 on an assumed capability would produce a feature that fails on first use with an opaque error; building it on the measured reality produces a feature whose default state is honest and whose enablement is one documented operator step.

Alternatives considered

- **Assume a generative model will be there by implementation time, and code for it.** Rejected: it makes the plan’s correctness contingent on an unrecorded future action, and there is no way to test the failure path if the happy path is the only one written.
- **Bundle a generative model into the repository.** Rejected outright: gigabytes of weights in git, in a repository that already had to split a 1.8 GB video into 36 parts to keep it out of git.
- **Reuse the resident jina-embeddings-v2-base-code for answering.** Rejected: it is an encoder with no LM head. It cannot generate. It is

however directly useful for the citation-verification step in §4 — see there.

- **Extractive-only answering (no generation at all): return the top passage verbatim as “the answer”.** Not rejected — **retained as the mandatory fallback mode.** It satisfies FR-021 (citation), FR-022 (it declines when nothing scores above threshold), FR-024 (nothing leaves the machine, no model involved), and FR-025 trivially. It cannot satisfy the *spirit* of US4 (a synthesised answer), but it means US4 delivers something real even with zero generative capability. See §3, provider extractive.
-

2. What HelixDevelopment / vasic-digital bridging already exists in this tree?

Decision

Nothing in this repository bridges to an LLM. Do not plan any dependency on HelixLLM, HelixAgent, LLMOrchestrator, or HelixMemory. Re-implement a minimal provider interface in this repository, modelled on the shape of the reachable LLMProvider module but not depending on it.

Evidence — what is present, and what it actually is

a) The name “HelixLLM” appears in this repository only as product marketing content.

```
git grep -rIl -E 'HelixLLM|HelixAgent|HelixDevelopment' -- .
```

Every hit is one of: `_content*/products/HelixLLM.md` (17 language variants), `_analysis/content-briefs/HelixLLM.md`, `_analysis/github-helix-others.md` (a repository inventory), `_content/docs/cv.md`, or `Constitution.md`. These are **portfolio-site copy describing products that live in other repositories**. `_analysis/content-briefs/HelixLLM.md:3` reads “*One binary, six modes — OpenAI- and Anthropic-compatible inference from your laptop to a multi-host cluster.*” That is a product description, not an integration.

b) There is zero LLM client code in this repository.

```
git grep -rIl -iE 'ollama|openai|anthropic|llm' -- . '(!_content*' '(!_analysis' '(!_*.json' '(!_submodules'
# → (no output)
```

The reference module's backend confirms it: `ai_interviewing/platform/backend/go.mod` requires `gin`, `cors`, `quic-go`, `goldmark`, `brothli` and `modernc.org/sqlite`. **No LLM client of any kind**. Its internal/ tree is `api/`, `ingest/`, `server/`, `store/` — a content server, not an AI service.

c) The CA-bundle / TLS wiring is real, but points at a path that does not exist.

```
env | grep -E 'SSL_CERT_FILE|NODE_EXTRA_CA_CERTS'
# SSL_CERT_FILE=<home>/.helixagent/ca-bundle.pem
# NODE_EXTRA_CA_CERTS=<home>/.helixagent/ca-bundle.pem
ls -l ~/.helixagent/ca-bundle.pem
# -rw-r--r-- 223959 Apr 30 11:39 (exists)
```

`~/ .bashrc:65-69` explains it: “*trust the self-signed HelixLLM cert via the unified CA bundle. The bundle contains system CAs + HelixLLM/certs/cert.pem (the live cert the HelixLLM server actually serves).*” But `~/ .profile:10` sets `NODE_EXTRA_CA_CERTS=<ext-volume-host>/Projects/HelixAgent/HelixLLM/certs`, and:

```
ls -d <ext-volume-host>/Projects/HelixAgent
# ls: cannot access ....: No such file or directory
```

The HelixLLM checkout the CA bundle was built to trust is not on this disk, and no HelixLLM server is listening (full `ss -ltn` sweep found nothing on any plausible HelixLLM port; only `127.0.0.1:11434` `ollama` and unrelated services). `.bashrc` wins over `.profile` for interactive shells, so the *live* value is the bundle file, which does exist — but the endpoint it authenticates does not.

Conclusion: the TLS plumbing is genuine and would work *if* a HelixLLM server were running. It is **named-only** as far as this feature is concerned.

d) ~/helix-agent exists as a binary — and it is NOT an LLM agent.

```
file ~/helix-agent
# ELF 64-bit LSB executable, x86-64, statically linked, Go
  BuildID=..., not stripped (23.7 MB)
~/helix-agent --version # → helix-agent version dev (build
  unknown)
~/helix-agent --help
# -bind-addr string      SWIM bind address (default "127.0.0.1")
# -bind-port int         SWIM bind port (default 7946)
# -etcd-endpoints string
```

```
# -wg-key string      WireGuard private key (base64)
# -wg-port int        WireGuard listen port (default 51820)
```

This binary is a **SWIM gossip / WireGuard / etcd cluster node**. It has no LLM flags, no model flags, no inference surface. Anyone reading HelixAgent in .bashrc and inferring an LLM bridge would be wrong. Recorded here specifically so nobody makes that inference later.

~/.helixagent/reports/provider_verification_report.md exists and says, verbatim: “**Status:** Verification service not yet initialized ... The verification report will be generated after the LLMsVerifier service completes provider verification.” Dated 25 Apr 2026. **It has never run.** There is no provider inventory to inherit.

e) OpenCode / kimi / mimo configs exist but declare no providers.

```
jq -r 'keys[]' ~/.config/opencode/opencode.json
# $schema instructions mcp skills
jq -r '.provider // {} | keys[]' ~/.config/opencode/
  opencode.json      # → (empty)
jq -r '.model // "-"' ~/.config/opencode/
  opencode.json      # → -
```

~/.config/opencode/opencode.jsonc and ~/.config/mimocode/mimocode.jsonc failed jq parsing (unescaped control characters / unterminated string) — **UNVERIFIED** whether they contain provider blocks; they were not repaired or read further, since doing so risks surfacing credential material. ~/.config/kimi* / ~/.kimi-code/mcp.json are MCP server lists, not LLM provider configs. None of these is a library this feature could call; they are configuration for *other agent CLIs*.

f) LLMPProvider — the one real, reachable artefact.

```
find <home> <ext-volume-host>/Projects -maxdepth 4 -type d -name
  LLMPProvider
# <home>/Yole/LLMPProvider
# <ext-volume-host>/Projects/helix_translate/LLMPProvider
git -C <ext-volume-host>/Projects/helix_translate/LLMPProvider
  remote -v
# origin git@github.com:HelixDevelopment/LLMPProvider.git
head -3 .../LLMPProvider/go.mod      # module
  digital.vasic.llmprovider ; go 1.25.3
ls .../LLMPProvider/pkg/providers | wc -l    # → 43
```

The 43 adapters include **ollama** and a **generic** OpenAI-compatible adapter:

ai21 anthropic cerebras chutes claude cloudflare codestral cohere
deepseek fireworks gemini

generic githubmodels groq huggingface hyperbolic junie kilo kimi
mistral modal nia nlpcloud
novita nvidia ollama openai openrouter perplexity publicai qwen
replicate sambanova sarvam
siliconflow together upstage venice vulavula xai zai zen zhipu

Its interface (provider.go) is exactly the seam this feature needs:

```
type LLMProvider interface {
    Complete(ctx context.Context, req *models.LLMRequest)
        (*models.LLMResponse, error)
    CompleteStream(ctx context.Context, req *models.LLMRequest)
        (<-chan *models.LLMResponse, error)
    HealthCheck() error
    GetCapabilities() *models.ProviderCapabilities
    ValidateConfig(config map[string]interface{}) (bool, []string)
}
```

And its ollama adapter already targets the right endpoints (pkg/
providers/ollama/ollama.go: http://localhost:11434 default, /api/tags
for health, /api/generate for completion).

Why it is still not usable as a dependency here:

```
cat ../LLMProvider/go.mod
# module digital.vasic.llmprovider
# require digital.vasic.models v0.0.0
# replace digital.vasic.models => ../Models
```

The module path digital.vasic.llmprovider is **not resolvable by the Go module proxy** — it is not a URL. It depends on a second unpublished module digital.vasic.models, wired by a **relative-path replace to ../Models**. Depending on it from vasic would mean depending on the *filesystem layout of a sibling project checkout* (helix_translate/), which is not in .gitmodules:

```
grep -E 'path|url' .gitmodules
# constitution, milosvasic.ru, vasic.digital, design-toolkit,
#   ai_interviewing,
# monetization, submodules/superspec, workshop      ← no
#   LLMProvider, no Models
```

Rationale

The distinction demanded — “exists here” versus “exists somewhere I cannot see” — resolves to a hard line:

Artefact	Status
HelixLLM (product)	

Artefact	Status
	Named only. Marketing copy in this repo; checkout absent from disk; no server listening.
HelixAgent (as an LLM bridge)	Named only, and the name misleads. The binary present is a SWIM/WireGuard cluster node.
LLMOrchestrator / HelixMemory	Named only. Listed in <code>_analysis/github-helix-others.md</code> ; not in <code>.gitmodules</code> ; not in this tree.
<code>~/ .helixagent/ca-bundle.pem + SSL_CERT_FILE</code>	Present and real , but authenticates an endpoint that does not exist here.
LLMProvider Go module	Real, reachable on this machine, outside this repository , unpublishable module path, relative-path dependency.
OpenCode / kimi / mimo provider config	Present, but declares no providers.
ollama at <code>127.0.0.1:11434</code>	Present, reachable, embedding-only. The one thing this feature can actually stand on.

Only the last row is a foundation. Everything above it is either absent, misnamed, or unbuildable from this tree. A plan that says “reuse HelixLLM” would be unverifiable, which FR-033 forbids.

The LLMProvider interface is nonetheless the right *shape*, and it is the operator’s own house style. Copying a five-method interface costs an afternoon; inheriting a relative-path dependency on a sibling checkout costs every future clone.

Alternatives considered

- **Add LLMProvider + Models as submodules under submodules/.** Rejected *for now*, recorded as the reopening condition. It would give 43 adapters for free and align with `helix-deps.yaml`. But it doubles the submodule surface for a feature that needs exactly two adapters, imports a module with an unresolvable path plus a

replace directive that a consuming build must also carry, and pulls in an unaudited transitive dependency set (`digital.vasic.models`) into a repository whose governance requires per-submodule carriers (G7 is already an open gap). **Reopens if:** the operator wants multi-provider failover, or if `LLMProvider` publishes a resolvable module path.

- **Vendor the ollama + generic adapters by copying the two directories.** Rejected: it copies code the operator maintains elsewhere, creating a silent fork with no upstream link — precisely the drift the constitution’s submodule mandate exists to prevent. If the code is wanted, take the submodule, not the copy.
 - **Call HelixLLM’s OpenAI-compatible endpoint over the CA bundle.** Not rejected as a *runtime* option — it is exactly what the `openai_compatible` adapter in §3 enables, and `SSL_CERT_FILE` already makes the TLS work. Rejected as a *design dependency*: no such server is running, so nothing about it can be tested here.
 - **Wait for the operator to start HelixLLM before finalising the design.** Rejected: it blocks Phase 0 on an external action. The provider seam in §3 makes HelixLLM a configuration value rather than a design premise, so nothing has to wait.
-

3. Provider abstraction — the seam (FR-023, FR-025)

Decision

A three-method Go interface with four adapters, selected entirely by a config file, defaulting to OFF. Reuse the resolution-order convention the repository already established in `scripts/lumen-reindex.sh`. Health is probed per request with a short timeout, and answering failures are architecturally incapable of reaching the search and browse paths.

```
// workshop/platform/backend/internal/answering/provider.go
type Provider interface {
    // Generate returns a completion. It MUST NOT retry across
    // providers.
    Generate(ctx context.Context, req Request) (Response, error)
    // Health is cheap, bounded, and side-effect free.
    Health(ctx context.Context) error
    // Describe is used by the /api/ask/status endpoint and by the
    // privacy gate in §5. Locality is DECLARED, never inferred.
```

```
Describe() Info // {Name, Endpoint, Model, Locality: Local|
  External}
}
```

Four adapters, all satisfying FR-023 between them:

Adapter	Endpoint shape	Covers
extractive	none — no model at all	The zero-generative-model fallback (§1). Always available.
ollama	POST /api/chat with format: <json schema>	The local default once a model is pulled.
openai_compatible	POST /v1/chat/completions	LM Studio (local), HelixLLM (local or internal), llama.cpp server, and every external vendor.
none	—	Explicitly disabled. The default.

Two adapters (ollama, openai_compatible) cover every real backend. openai_compatible is what makes “use all technology we have in HelixDevelopment” true *without* a code dependency: HelixLLM’s own brief says it serves OpenAI-compatible APIs, and SSL_CERT_FILE already trusts its cert — so pointing the config at it is a one-line change with zero new code.

Configuration surface

workshop/platform/config/answering.yaml, every key overridable by an environment variable, resolved in the **same order scripts/lumen-reindex.sh already uses** (env → config file → live probe → documented fallback):

```
answering:
  enabled: false                # DEFAULT OFF
  provider:
    none                        # none | extractive | ollama |
    openai_compatible
  endpoint: "http://127.0.0.1:11434"
  model: ""                     # empty ⇒ resolved by live
                                probe, see below
  locality:
    local                       # local | external – DECLARED,
                                enforced in §5
```

```

api_key_env: ""                # NAME of an env var. Never a
                               value. Never a file path.
timeouts:
  health_seconds: 2
  generate_seconds: 240        # see §6 – this is not a typo
retrieval:
  top_k: 8
  min_score: 0.0              # calibrated, see §4. 0.0 ⇒
                               uncalibrated ⇒ refuse everything
  min_margin: 0.0
verification:
  mode: strict                # strict | identifier_only

```

Env overrides: WORKSHOP_ANSWERING_PROVIDER, WORKSHOP_ANSWERING_ENDPOINT, WORKSHOP_ANSWERING_MODEL, WORKSHOP_ANSWERING_LOCALITY, and — deliberately mirroring the existing convention so an operator learns one pattern — OLLAMA_HOST is honoured as a fallback for endpoint when provider: ollama.

Model resolution when model: "" copies lumen-reindex.sh:129-171 exactly: try env, then config, then /api/ps (what is already loaded), then /api/tags (what is available), then a documented fallback. On this machine today, that walk lands on an embedding model, and the adapter **must reject it**: an ollama adapter that sees family: jina-bert-v2 and no generative capability must fail construction with "model X is an embedding model, not a generative model" rather than sending a /api/chat request that returns nonsense. This is a named check because it is the exact bug this environment invites.

Defaults, and why they are what they are

- enabled: false, provider: none. **A fresh clone answers nothing.** It therefore cannot leak (FR-024 vacuously), cannot fabricate (SC-010 vacuously), and *must* keep search and browse working (FR-025) because that is the only mode it has. The out-of-the-box state is the safe state.
- locality is **declared, not inferred from the endpoint string.** Inferring "127.0.0.1 means local" is a string comparison an attacker or a typo defeats (localhost.example.com). Declaring it means the privacy gate in §5 has something unambiguous to enforce against, and a mismatch between declaration and reality is a *detectable* fault rather than a silent one.
- api_key_env holds the **name** of an environment variable. The value is never written to config, never logged, never included in the Describe() output, and never echoed in the status endpoint. ~/api_keys.sh (mode 600, sourced by ~/.bashrc:51) is the machine's

existing credential store; the config references a variable name it exports, and nothing in this feature ever reads that file.

- `min_score: 0.0` as the shipped default is deliberately *hostile*: an uncalibrated threshold refuses everything (see §4). The operator must run the calibration gate to get a real number. This makes “we never calibrated” fail loudly instead of silently degrading to “answer everything”.

Behaviour when the provider is unreachable (FR-025)

Three states, matching FR-033 / SC-013 exactly — and note that these are *the same three states* the answering pipeline uses in §4:

State	Meaning	HTTP	UI
answered	Answer produced, every citation verified	200	Answer + citations
declined	Content genuinely does not support an answer	200	“The indexed content does not answer this.” + the passages that were closest
unavailable	Provider not configured / unreachable / verification could not run	503 on /api/ask only	“Answering is unavailable.” Search box stays live.

declined returns **200, not an error** — declining is a correct result, and conflating it with a failure would make SC-010 unmeasurable.

Architectural enforcement of FR-025, in order of strength:

1. **Separate route trees.** `/api/search`, `/api/suggest`, `/api/chapters`, `/api/passages` are registered on a router group that has no reference to the answering package. `/api/ask` and `/api/ask/status` are the only routes that construct a Provider. A compile-time guarantee beats a runtime one.
2. **No shared initialisation.** Provider construction failure must not abort server startup. The server starts with `provider = unavailableProvider{reason: err}` and serves everything else.

3. **Bounded health probe before commit.** `health_seconds: 2`, checked before any generation is started, so an unreachable provider costs 2 s and not `generate_seconds`.
4. **No shared goroutine pool or connection pool** between the answering client and the search path — otherwise a hung 240 s generation starves search and SC-006 (2 s p95) fails for a reason unrelated to search.
5. **The gate that proves it** (FR-032): stop ollama (or point endpoint at a closed port), assert `/api/search` returns 200 with real results and `/api/ask` returns 503 with `state: unavailable`. **Paired mutation:** wire search through the provider's health check and assert the gate now reports FAIL. Without the mutation the gate proves nothing.

Rationale

The seam has to satisfy two things simultaneously: an operator switching local↔external with no code change (FR-023), and a *demonstrable* guarantee that answering cannot take search down (FR-025). Configuration alone gives the first; only route separation gives the second, because any shared object is a shared failure mode. Defaulting to off converts the hardest requirements into their trivially-satisfied form for anyone who has not opted in, which is the correct risk posture for a feature whose failure mode is confident falsehood.

Alternatives considered

- **Environment variables only, no config file.** Rejected: the retrieval thresholds in §4 are calibrated numbers that must be version-controlled and reviewable alongside the calibration evidence (FR-040). A number that lives only in a shell export cannot be reviewed at the commit that produced it.
- **Auto-detect the provider by probing common ports at startup.** Rejected: it makes behaviour depend on what happened to be listening, which is unreproducible, and it would silently route content to whatever answered — a privacy hazard under FR-024.
- **Infer locality from the endpoint hostname.** Rejected as described above; string matching on hostnames is not a security boundary.
- **Cross-provider failover (try local, fall back to external).** **Rejected firmly.** It is the single most dangerous convenience feature available here: a local-only deployment would silently start shipping private workshop content — a recording of an identifiable third party (spec D1) — to an external vendor the moment the local

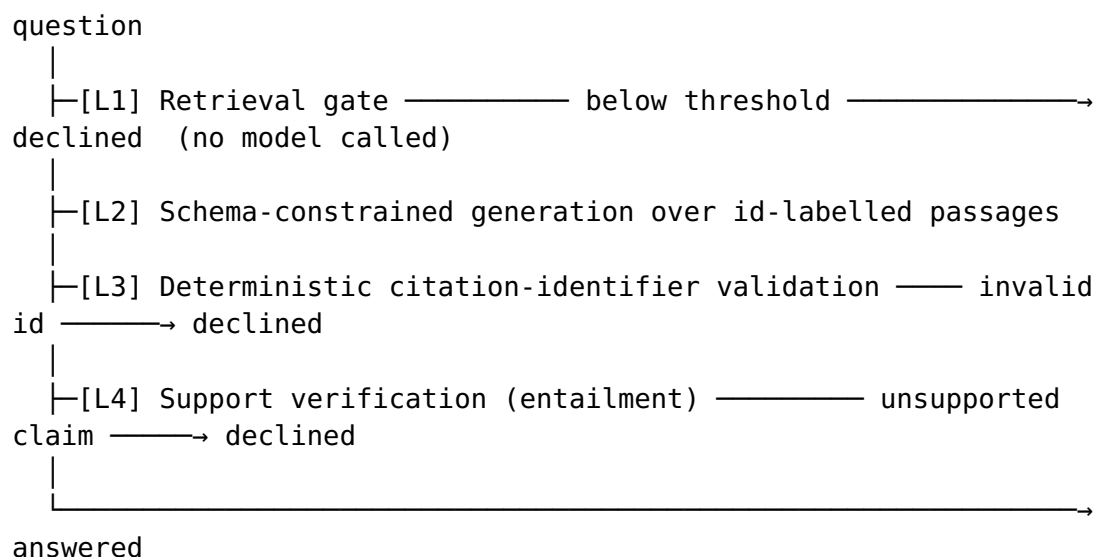
model hiccuped. FR-024 is violated by exactly this. Failover may only ever occur *within* a declared locality, and is out of scope.

- **Streaming responses (CompleteStream).** Deferred, not rejected. §6 shows a first token can be tens of seconds away, so streaming materially improves perceived latency. But the verification pass in §4 can only run on a *complete* answer — you cannot verify a citation for a sentence that is still being written. Streaming would mean showing the user text that has not yet passed verification, which risks displaying an unsupported claim for several seconds. **If streaming is added later, the verified/unverified state must be visible in the UI throughout, and the answer must not be presentable as final until verification completes.**
-

4. Grounding and refusal — the core (FR-021, FR-022, SC-009, SC-010)

Decision

A four-layer pipeline in which each layer catches a failure the others cannot, and in which the deterministic layers do the heavy lifting. Refusal is decided by a combination — never by a threshold alone, never by an instruction alone, never by a verifier alone. Any layer’s failure produces a refusal, never a partial answer.



L1 — Retrieval gate: refuse before the model is ever called

Retrieve top_k passages from the existing index. Refuse *before generation* if either holds:

- $\text{score}(\text{top1}) < \text{min_score}$ — nothing is close enough.
- $\text{score}(\text{top1}) - \text{mean}(\text{score}(\text{top2}..\text{topk})) < \text{min_margin}$ — nothing *stands out*; the query matched the corpus diffusely, which is what an off-topic question looks like.

min_score and min_margin must be calibrated, not guessed. Raw cosine similarity from jina-embeddings-v2-base-code is not calibrated: unrelated text pairs routinely score 0.5–0.7, so a hand-picked “0.7 looks about right” is a coin flip dressed as engineering. Calibration procedure, run as a gate whose output lands in the versioned evidence directory (FR-040):

1. Take the SC-007 benchmark (≥ 20 answerable queries with known target passages) and the SC-010 set (≥ 10 unanswerable, constructed as below).
2. Record $\text{score}(\text{top1})$ and the margin for every query in both sets.
3. Choose the threshold at **the lowest value that refuses 100% of the unanswerable set**, then report what fraction of the answerable set that costs.
4. **Publish both numbers.** If the threshold that achieves 100% refusal also refuses 40% of answerable questions, that is the honest trade and it must be visible — not hidden behind a threshold tuned to make both look good on the same 30 examples.
5. Re-run the calibration whenever the corpus or the embedding model changes; a stale threshold is a silent regression.

The margin test matters more than it looks. The dangerous unanswerable question is not “what is the capital of France” — it is “what Docker version did Milos say he was running?” when Docker *is* discussed at length. That question scores **high** on min_score and **low** on margin, because many passages mention Docker and none answers the question. The two tests fail on different questions, which is the entire reason both exist.

L2 — Generation: bind citations mechanically, not by request

Passages go into the prompt each labelled with its **persisted stable identifier** (FR-037), and the model is required to emit a structure whose grammar cannot express an uncited claim:

```
[P:0f3c9a1e] (transcript, ch01, 00:14:32–00:15:10, speaker: Milos, confidence: high)
"...the actual text of the passage..."
```

```
[P:7b21d4c0] (docs, chapter-01/setup.md §3)
"..."
```

Output is constrained by JSON schema — ollama 0.23.4 (verified above) supports format with a JSON schema, and the openai_compatible adapter uses `response_format: {type: json_schema}`:

```
{
  "type": "object",
  "required": ["answerable", "claims"],
  "properties": {
    "answerable": {"type": "boolean"},
    "claims": {
      "type": "array",
      "items": {
        "type": "object",
        "required": ["text", "citations"],
        "properties": {
          "text": {"type": "string"},
          "citations": {
            "type": "array", "minItems": 1,
            "items": {"type": "string", "pattern": "^[0-9a-f]{8}$"}
          }
        }
      }
    }
  }
}
```

"minItems": 1 is the load-bearing line. **The decoder cannot emit a claim with zero citations.** This converts “please remember to cite” — a request a 3B model will sometimes ignore — into a constraint it cannot violate. It does *not* make the citation correct; that is L3 and L4’s job. It makes an *uncited* claim structurally impossible, which removes one whole failure class for free.

Generation runs at temperature: 0, seed fixed, so SC-009 and SC-010 results are reproducible and a passing run can be re-run by a reviewer.

The instruction states plainly: answer only from the numbered passages; do not use outside knowledge; if the passages do not contain the answer, set `answerable: false` and return no claims. This instruction is **layer two of four, and is assumed unreliable** — small instruction-tuned models are exactly the models most prone to answering anyway. It is included because it is free and it helps; it is never trusted alone.

L3 — Deterministic identifier validation (this is what makes SC-009 reachable)

For every citation id emitted:

- **Is it in the set of ids actually placed in this prompt?** If not, the model invented an identifier. **Refuse the entire answer.** This is a string-set membership test — deterministic, instant, and it catches the single most common citation failure with certainty.
- **Does it resolve to a live passage in the store?** (Guards against a redacted passage — FR-039 — being cited from a stale in-flight prompt.)
- **Is the passage still the one the id names?** FR-037's persisted identifiers make this a lookup, not a heuristic.

L3 costs microseconds and is the highest-value layer per unit of cost in the whole design. **A pipeline without L3 cannot claim SC-009 at all**, because “the citation points at a passage that exists” is the floor beneath “the citation supports the claim”.

L4 — Support verification: is the citation *doing its job*?

L3 proves the citation is real. L4 proves it is *relevant*. SC-009 says 100% of citations must “genuinely support the claim” — an attached-but-unsupporting citation fails SC-009 just as hard as a fabricated one, and L1–L3 cannot detect it.

Two stages, cheap first:

L4a — semantic floor (cheap, uses the model that is already installed). Embed the claim text and each cited passage with `ordis/jina-embeddings-v2-base-code` — the model that is *already resident on this machine and already serves the search index*. If a claim's similarity to *every* passage it cites falls below a calibrated floor, the citation is

decorative. Reject without spending a generation. Measured cost: **~0.10 s per embed call on an idle backend** (§6), so a five-claim answer costs well under a second. This is a *necessary* condition, not a sufficient one — high similarity does not prove entailment (a passage saying “we did NOT use Kubernetes” is highly similar to a claim saying “we used Kubernetes”).

L4b — entailment check (the sufficient condition). One additional constrained call to the same provider, batched over all claims at once:

For each numbered (claim, passage) pair, does the passage state or directly imply the claim? Answer only supported or not_supported.

Output schema: `{"verdicts": [{"claim_index": int, "verdict": "supported" | "not_supported"}]}`. Batching matters for cost: the passages are the bulk of the prompt and they are **already in the KV cache from L2**, so L4b is roughly one extra *prefill of the claims* plus a handful of output tokens — call it **~1.3–1.6× the L2 cost, not 2×** (**UNVERIFIED**: no generative model installed to measure; see §6).

`verification.mode: identifier_only` exists as a config option for a deliberately-degraded fast mode — **and when it is set, the system must state in its response that citations are unverified, and the SC-009 gate must refuse to report PASS**. A mode that quietly skips the check while the success criterion still claims 100% would be exactly the bluff the constitution forbids.

On any failure, refuse the whole answer. Do not silently drop the failing claim and return the rest. Dropping claims produces an answer that reads as complete but has had its qualifications removed — a *new* correctness surface, and a worse one, because the reader cannot see that anything was removed.

Why the combination, and not any single mechanism

This is the question the topic asks to be justified, so it is answered directly:

Mechanism alone	The failure it cannot catch
Retrieval threshold only	The near-miss. Entity present, attribute absent. Scores high, answers nothing. The model, handed a high-scoring passage

Mechanism alone	The failure it cannot catch
	and asked a question it does not answer, fills the gap.
Instruction only	Weak instruction-following. The models this box can run at usable speed are 1.5B–3B — the size class most likely to answer anyway, especially with a plausible-looking passage in context.
Schema constraint only	Guarantees a citation is <i>attached</i> . Says nothing about whether the id is real (L3) or relevant (L4). Produces confident, well-formatted, wrong answers.
Verification only	Pays full generation cost on every unanswerable question, and the verifier is the <i>same weak model</i> that just produced the claim — self-verification by a small model is not independent evidence. Also has no cheap path when the retrieval was hopeless from the start.

Each layer’s blind spot is another layer’s specialty. And the ordering is chosen so the **deterministic, free layers (L1, L3) carry the load** and the expensive probabilistic layer (L4b) runs only on answers that already survived them.

Constructing the ≥ 10 unanswerable questions so SC-010 means something

The trap: ten questions about astrophysics would pass any threshold and prove nothing. A meaningful set must be *adversarially near* the corpus — questions a naive system would answer. Required taxonomy, at least one of each, ≥ 10 total, all about Chapter 1:

1. **Near-miss attribute** — entity present, asked attribute absent. “*What version of Docker did Milos say he was running?*” when Docker is discussed and no version is stated.
2. **Adjacent topic** — plausibly in the workshop’s domain, never covered. “*How does the team rotate API credentials?*”

3. **False-premise / counterfactual** — presupposes something the transcript contradicts. “*Why did Milos recommend against using Claude Code?*” when he recommended for it. Tests whether the system corrects the premise or invents a justification for it — the highest-value single test in the set.
4. **Out of temporal scope** — “*What is covered in Chapter 2?*” The corpus has one chapter (spec Context table).
5. **Uncomputable aggregate** — “*How many minutes were spent on each topic?*” Derivable-looking, not derivable.
6. **Precise figure with no figure present** — “*What latency number did Milos quote for the build?*” Invites fabrication of a plausible number, which is the most damaging failure mode because it is the least detectable by a reader.
7. **Misattributed speaker** — “*What did the second speaker say about X?*” where X was said only by Milos. Ties directly to FR-005.
8. **Lexically overlapping but unanswerable** — built from vocabulary that is *frequent* in the transcript, phrased as a question the transcript does not answer. Specifically designed to defeat `min_score` and be caught only by `min_margin`.
9. **Redacted content** — asks about a passage suppressed under FR-039. Must decline, and must not answer from a stale index entry. Ties SC-010 to FR-039.
10. **Inaudible segment** — asks about content inside a span marked uncertain under FR-003. The correct behaviour is to decline (or to say the audio was unclear) — never to guess at what was probably said.

Three rules that keep the test set from becoming a bluff:

- **Every question carries a human certification** that it is genuinely unanswerable from the transcript, recorded as a signed line in the fixture next to the question. A test set nobody verified is an assertion, not evidence.
- **The gate records `score(top1)` and the margin for every one of the ten.** A question that refuses because it landed 0.002 below the threshold is *fragile*, and reporting it as a comfortable pass would be dishonest. The evidence file shows the distance from the threshold, not just the verdict.
- **Paired mutation (FR-032, SC-012).** Seed a fixture that disables L1 (set `min_score: 0.0`) and assert the SC-010 gate reports **FAIL**. Then seed one that makes L3 accept unknown ids and assert **FAIL** again. A gate with no demonstrated failure mode is not a gate.

Honest limitation, stated rather than glossed: ten questions is a floor, and 10/10 is weak statistical evidence — it is consistent with a true refusal rate anywhere above roughly 75%. Fixed temperature: 0 and a fixed seed make the result *reproducible*, which is what makes it reviewable, but reproducible is not the same as *generalising*. **The gate must report “10/10 declined, deterministic run, scores recorded” and must not be paraphrased as “the system never fabricates.”** Growing the set beyond ten is the cheapest available improvement to confidence and should be a standing task.

Rationale

SC-009 demands **100%**, which is a threshold no probabilistic pipeline reaches by being well-prompted. It is reachable only if the deterministic checks carry the guarantee: L3 makes fabricated identifiers *impossible to return*, and L4 makes unsupported ones *detected before display*. The generation step is then free to be imperfect, because nothing it produces is displayed without surviving checks that do not depend on it being right. That inversion — deterministic verification gating a probabilistic generator, rather than a probabilistic generator being trusted — is the whole design.

Alternatives considered

- **Threshold-only refusal.** Rejected: the near-miss class (taxonomy item 1) is the *common* case in a single-chapter corpus where everything is topically related, and threshold-only is blind to it.
- **Prompt-engineering only (“say I don’t know if unsure”).** Rejected: unverifiable, model-dependent, and it degrades exactly when the model is small — which is the only kind this hardware runs fast enough to use.
- **A dedicated NLI model (e.g. DeBERTa-MNLI) for L4b.** Genuinely attractive — cheaper and more reliable per check than an LLM, and small enough to run fast on CPU. **Rejected for now** because it means a second model download and a second serving path, and the primary blocker (§1) is that even the *first* model is not installed. **Reopens** as the recommended L4b upgrade once a generative model exists and L4b’s cost is measured to be the bottleneck.
- **Lexical overlap (ROUGE / n-gram containment) as the whole of L4.** Rejected as sufficient: it is defeated by correct paraphrase (false negatives) and by negation (false positives). Retained as a *supplementary* cheap signal alongside L4a.

- **Returning a partial answer with failing claims stripped.**
Rejected — see L4. Silently removing qualifications produces a more confident, less true answer.
 - **Answer caching.** Deferred. It would help enormously with §6’s latency, but a cached answer must be invalidated when any cited passage is corrected (FR-038) or redacted (FR-039) — spec edge case “*A passage is redacted after it has been indexed and cited*” names this explicitly. Cache keyed on (question, model, set of cited passage ids + their version), invalidated on passage change. **Not in the first release:** an answer cache that misses an invalidation serves a redacted passage, which is worse than being slow.
-

5. Privacy enforcement (FR-024) — enforced and demonstrated, not configured

Decision

Four layers, of which the third is the only one that constitutes proof: a resolved-address allowlist, a declared-locality invariant, an egress-denied network namespace with a negative control, and a packet-level capture assertion over the full SC-009/SC-010 test runs — all written to the versioned evidence directory, with a paired mutation proving the gate can fail.

A config flag is not a guarantee. `locality: local` is a *declaration*; FR-024 requires that the declaration be *true*, and truth here means “the process could not have sent content out even if it tried”.

L1 — Resolved-address allowlist (construction-time)

When `locality: local`, the provider constructor resolves endpoint’s host with `net.LookupIP` and requires **every** returned address to satisfy `ip.IsLoopback()`. Not a string comparison — `localhost.evil.example` contains the substring `localhost` and resolves anywhere. Multi-address hosts must be loopback on *all* addresses; one non-loopback A record and construction fails.

Refuse to construct (not “warn”) if: any address is non-loopback; the scheme is not `http/https`; or `api_key_env` is set (a local endpoint needing a vendor API key is a contradiction worth failing on).

L2 — Declared-locality invariant

`Describe().Locality` is surfaced by `/api/ask/status` and rendered in the UI. A gate asserts that when `locality: local`, `Describe().Endpoint` resolves to loopback — i.e. that the declaration and the resolved reality agree. Divergence is a **fault**, not a warning.

L3 — Egress denial with a negative control (this is the proof)

Run the answering service inside a network namespace whose only interface is `lo`, with `ollama` in the same namespace. The plan already targets `podman` (plan.md: “*podman + podman-compose; docker is absent*”), so this is available without new infrastructure.

The gate has two halves, and **the negative control is the half that makes it evidence**:

```
# NEGATIVE CONTROL – egress must be impossible
ip netns exec workshop-answering \
    curl -s --max-time 5 https://example.com
    # MUST fail (exit != 0)
ip netns exec workshop-answering \
    getent hosts example.com                                # MUST fail (no
    resolver reachable)

# POSITIVE CONTROL – the feature must still work
ip netns exec workshop-answering \
    curl -s --max-time 10 http://127.0.0.1:11434/api/tags # MUST
    succeed
```

The negative control is what upgrades “we observed no egress” to “egress was impossible”. Without it, a passing run only proves the system *did not happen to* send anything during the observation window.

L4 — Packet-level capture over the real test runs

Run the full SC-009 sample (≥ 20 answers) and the SC-010 set (≥ 10 refusals) with a capture attached to the namespace:

```
ip netns exec workshop-answering \
    tcpdump -i any -w evidence/fr-024-egress.pcap 'not host
    127.0.0.1 and not ip6 host ::1'
```

Assertion: zero captured packets. Written to `workshop/_tests/evidence/<commit>/fr-024/` per FR-040, retained with the commit that produced it: the pcap, its packet count, the exact capture filter, the

namespace name, and the resolved endpoint. The filter is recorded because a capture with an over-broad filter proves nothing and a reviewer must be able to see the filter that was actually used.

Paired mutation (FR-032 / SC-012) — proving the gate can fail

Three mutations, each asserting the gate reports **FAIL**:

1. Set locality: `external` with endpoint pointing at a stub HTTP listener on the host's **LAN** address (192.168.1.44, observed in `ss` output). Run one question. The gate must catch the packets and **FAIL**.
2. Set locality: `local` but endpoint: `http://192.168.1.44:11434`. L1 must refuse construction; the gate must **FAIL** if it does not.
3. Remove the namespace and run on the host network with the same stub. L3's negative control must now *succeed* in reaching outside, and the gate must **FAIL** because the control it depends on has stopped controlling.

Mutation 3 is the important one: it guards against the gate silently passing because the namespace was never created.

Supporting measured fact — ollama's own binding

```
ss -ltn | grep 11434
# LISTEN 0 4096 127.0.0.1:11434 0.0.0.0:*
```

ollama binds loopback **only**. A future `OLLAMA_HOST=0.0.0.0` would silently widen this to the LAN, so a gate asserts the bind address remains loopback. Recorded as a measured fact today, and as a standing check because measured-once is not the same as guaranteed.

Rationale

FR-024 is a *negative* requirement — “MUST NOT transmit” — and negatives cannot be proven by observing correct behaviour once. They are proven by making the prohibited action impossible and then demonstrating the impossibility (the negative control) alongside the absence (the capture). The four layers form a chain where each earlier layer makes the later one cheaper to satisfy, and only L3+L4 together constitute evidence a reviewer can check.

This matters more here than in a generic system. Spec D1 records that the recording is “*a private teaching session between two named individuals*” and that publishing it “*creates a privacy exposure that cannot be undone*”. The content being protected is a third party’s voice and likeness, and the person exposed by a leak did not choose this system’s configuration.

Alternatives considered

- **Trust the config flag.** Rejected: it is precisely the insufficient design the research topic names. A flag records an intention; FR-024 constrains an outcome.
 - **--network=none.** Rejected: it also severs loopback-to-host, so the ollama endpoint becomes unreachable and the feature cannot run at all.
 - **slirp4netns / user-mode networking.** Rejected: it still permits outbound traffic. It changes the path, not the possibility.
 - **Host-level nftables/iptables rules matching the service’s UID or cgroup.** A viable alternative to a namespace, and slightly less invasive. **Rejected as primary** because it mutates host firewall state — the constitution requires a hardlinked backup before destructive operations, and a firewall rule that outlives the test is a change to the operator’s machine that the test did not advertise. **Retained as the fallback** if namespaces are unavailable in the target runtime; the same negative control applies unchanged.
 - **Static analysis (assert no outbound HTTP client is constructed outside the provider package).** Useful and cheap, and worth adding as a lint. Rejected as *sufficient*: it cannot see a dependency’s egress, and FR-024 constrains runtime behaviour, not source shape.
-

6. Cost and latency — the honest number

Decision

“Instant answers” is not achievable on this hardware and must not be promised. Plan for tens of seconds per answer, present it as an explicitly asynchronous interaction, and hold the “instant” experience where it is actually achievable — search and autocomplete (SC-005 ≤ 200 ms, SC-006 ≤ 2 s), which do not involve generation.

Measured: the backend is a single queue, and it is currently saturated

Three identical requests, same model (resident, per /api/ps), same two-word payload, minutes apart:

```
curl -s --max-time 60 http://localhost:11434/api/embed \
-d '{"model":"ordis/jina-embeddings-v2-base-code","input":"hello world"}' \
-o /dev/null -w 'http=%{http_code} total=%{time_total}s\n'
```

Run	Result
1	http=200 total=20.161822s
2	http=200 total=11.050511s
3	http=200 total=0.104562s

```
uptime    # 22:20:24 up 10:21, load average: 8.25, 8.12, 7.87
```

Load average 8.25 on 8 logical CPUs — fully saturated by the concurrent lumen index --force rebuild and the 287-gate sweep. The 200× spread on an identical call is not compute variance; it is **queue wait**. Run 3 (0.10 s) is the true unloaded cost of a small embedding; runs 1 and 2 are that same work waiting behind indexing batches.

This is the single most actionable measurement in this document, and it holds regardless of which generative model is eventually installed:

- The **true idle** cost of an embedding is ~0.10 s. L4a (\$4) is genuinely cheap.
- Under concurrent indexing, **the same call takes 11–20 s**. ollama serves one queue; generation and indexing contend directly.
- Design consequences, all mandatory:
 1. **The chapter-ingest procedure (FR-026) must take an exclusive lock that puts answering into unavailable for its duration.** This satisfies FR-025 correctly — search continues from the *existing* index — and it is far better than letting answers take four minutes and appear hung.
 2. **A separate ollama instance on a second port for answering** is the alternative if concurrent operation is ever required. Not recommended initially: two instances double the resident memory and still contend for the same four cores.
 3. **The UI must never present a spinner with no state.** Show the provider state, the elapsed time, and a cancel control. The spec's

own edge-case list already records a 75 s search timeout caused by a saturated embedding backend — this failure has happened here before.

Estimated: CPU-only generation latency (UNVERIFIED — no generative model is installed)

These are engineering estimates, not measurements, because §1 establishes there is nothing to measure. The method is stated so the numbers can be checked and replaced with real ones the moment a model is pulled.

Token generation on CPU is **memory-bandwidth bound**: tok/s $\approx$ achievable_bandwidth / model_bytes. This box is a 4-core Tiger Lake with LPDDR4x — roughly 60 GB/s theoretical, realistically ~**30–40 GB/s** achieved by llama.cpp. Prompt prefill is **compute bound** and benefits from the AVX-512/VNNI confirmed present.

A grounded answer under §4 involves a prompt of roughly **1 500–2 500 tokens** (top_k: 8 passages of ~200–300 tokens each, plus instructions and schema) producing ~**150 output tokens**.

Model (Q4_K_M)	Prefill (tok/s)	Decode (tok/s)	2 000- token prefill	150- token decode	L2 total	+ L4b (~1.4×)
1.5B	~120–180	~15–25	~13 s	~8 s	~ 21 s	~ 30 s
3B	~60–100	~8–12	~25 s	~17 s	~ 42 s	~ 59 s
7B	~25–45	~3–5	~57 s	~38 s	~ 95 s	~ 133 s

On an idle machine. Under the contention measured above, add the queue wait — which was observed at 11–20 s for a *trivial* call, and would be substantially larger for a generation request.

Add to every row: retrieval (~0.1–0.3 s, embedding + index lookup) and L4a (~0.1 s per claim). Both are negligible against generation. **The generation step is ~99% of the answer latency**, which is why generate_seconds: 240 in §3 is a realistic timeout and not a typo.

The honest bottom line: 20–130 seconds per grounded answer on this hardware, idle. Under concurrent indexing, longer, with no useful upper bound.

The conflict with the operator’s request, stated plainly

The feature request asks for “asking the AI questions and getting instant answers” and “everything ran instantly and maximal UX achieved”. Measured against this hardware:

- **Search and autocomplete can be instant.** SC-005 (≤ 200 ms p95) and SC-006 (≤ 2 s p95) are achievable — they are index lookups over pre-computed embeddings, and the 0.10 s idle embedding measurement supports it. **This is where the “instant” experience lives, and US3 delivers it.**
- **Generated answers cannot be instant.** Not with better prompts, not with a smaller model — a 1.5B model is still ~ 21 s, and it is the size class least able to satisfy SC-010. The gap between 21 s and “instant” is two orders of magnitude, and no amount of engineering on four CPU cores closes it.

The options, honestly:

Option	Latency	Consequence
Accept asynchronous answering	20–130 s	Recommended. Answering is framed as “ask and get a verified answer shortly”, not as chat. Search stays instant.
Use a 1.5B model	~ 21 –30 s	Fastest, but weakest instruction-following — the highest SC-010 risk. Requires the strictest L1 threshold.
Configure an external provider	1–5 s	Fast, and FR-023 explicitly supports it. But locality: external means workshop content leaves the machine, which spec D1 and FR-024 exist to prevent. An operator decision with a privacy cost, never a default.

Option	Latency	Consequence
Enable the GPU	faster	Rejected. scripts/ollama-vulkan-remediation.sh records that Vulkan on this iGPU <i>silently corrupted embeddings</i> . Trading correctness for speed here is the exact bargain this repository already refused once. Genuinely instant, genuinely grounded (it returns the passage itself), cannot fabricate. Not a synthesised answer.
extractive provider (§1)	~0.3 s	A good default for the impatient path and the zero-model state.

Rationale

The plan needs the true number. Writing “answers in a few seconds” into a plan that will produce 60-second answers guarantees the feature is judged a failure on first use, and the failure will be attributed to the implementation rather than to the hardware. Stating 20–130 s up front lets the design absorb it: an asynchronous UI, an ingest lock, an extractive fast path, and an external-provider escape hatch the operator can choose *knowingly*. It also protects US1–US3, which deliver the operator’s “instant” ambition in the place where it is actually reachable.

Alternatives considered

- **Quote the desired latency and hope.** Rejected — this is the bluff the constitution names.
- **Speculative decoding / draft models.** ~1.5–2× decode speedup at the cost of a second resident model and considerable complexity.

Not worth it before the base case is measured; revisit only if the 3B path proves close to acceptable.

- **Shrink the prompt (top_k: 3 instead of 8) to cut prefill.** Real and significant — prefill is ~60% of the total, so this is the single largest available saving. **Rejected as a default** because it directly reduces recall, which pressures both SC-007 and (worse) L1’s ability to distinguish “answerable” from “unanswerable”. Correctness before speed. top_k is configurable, so an operator can make the trade knowingly.
- **Warm the model with keep_alive.** Adopted, not an alternative — set keep_alive so the generative model stays resident between questions. It removes model-load time from every request after the first. Note the memory cost is additive to the resident embedding model, which is fine given 53 GiB free.
- **Answer caching.** See §4 — deferred for invalidation-correctness reasons, not cost reasons.

Consolidated decision list

#	Decision	Drives
D-LLM-1	No generative model exists; US4’s default state is answering unavailable	FR-025, US1–US3 sequencing
D-LLM-2	Do not depend on HelixLLM/ HelixAgent/ LLMProvider; re-implement a 3-method interface	FR-023
D-LLM-3	Four adapters — none (default), extractive, ollama, openai_compatible	FR-023, FR-024
D-LLM-4	Config-file seam, enabled: false default, locality declared not inferred	FR-023, FR-024
D-LLM-5	Separate route trees; answering cannot	FR-025

#	Decision	Drives
	share failure paths with search	
D-LLM-6	Four-layer grounding: calibrated retrieval gate → schema-constrained generation → deterministic id validation → support verification	FR-021, FR-022, SC-009, SC-010
D-LLM-7	Refuse whole answers on any verification failure; never return partial	SC-009
D-LLM-8	Ten-question unanswerable set built from a 10-item adversarial taxonomy, human-certified, scores recorded	SC-010
D-LLM-9	Privacy proven by egress-denied namespace + negative control + packet capture + paired mutation	FR-024, FR-032, FR-040
D-LLM-10	Ingest takes an exclusive lock that suspends answering; search continues from the existing index	FR-025, FR-026
D-LLM-11	Honest latency: 20–130 s per answer idle; “instant” belongs to search, not answering	SC-005, SC-006

Open items this research could not close

Item	Why it is open
Actual generation latency	UNVERIFIED — no generative model installed; pulling one is forbidden by this task's constraints. Replace §6's table with measurements immediately after the operator pulls a model.
min_score / min_margin values	Cannot be calibrated before the transcript exists (US1). Calibration is a gate, not a constant.
L4b cost multiplier (~1.4×)	UNVERIFIED — depends on KV-cache reuse behaviour in the chosen model and ollama version.
Whether Lumen chunk ids are stable across re-index	Owned by research/search-architecture.md. If they are not stable, FR-037 needs its own identity layer, and L3 in §4 depends on it — this is a direct cross-dependency, not an aside.
opencode.jsonc / mimocode.jsonc contents	UNVERIFIED — both fail JSON parsing; not repaired or read further to avoid surfacing credential material.
Provider-side behaviour of HelixLLM	UNVERIFIED — no HelixLLM server is running and no checkout exists on this disk. It is reachable through the openai_compatible adapter <i>if</i> the operator starts one; nothing about it was tested.